EPSR-test-ex55 voids: two void distributions, fresh random numbers

Program(s): voids | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises `voids`, which maps the cavities of a configuration [1]: the box is divided into cubic pixels, pixels within `radius` of an atom are occupied, connected unoccupied pixels are labelled as voids, and the void-size histogram, the pair correlation of void pixels (and its inter-void part) and the void volume fraction are written. This case tests a single run with two distributions, the `include` list and a reset random-number generator.

Code exercised

Fortran (original, fortran_src/):

- `voids.f`: `run_voids`, `find_voids`, `occupypix`, `markpixel`, `search_voids`, `get_void_list`, `ranmine`
- `auxiliary_routines.f`: `initran1`, `ran1`, `saveran1` (generator state in `randomno.dat`)
- `read_and_save_files.f`: `outgofz`, `save_files_others`; `setup_symbols_generic.f`: `setup_symbols_voids`

Rust (port, rust/epsr-core/src/):

- `tools/voids.rs`: `run`, `find_voids`, `occupypix`, `markpixel`, `search_voids`, `get_void_list`
- `rng.rs`: `Ran1::ran1`; `tools/anal_common.rs`: `initran1_file`, `saveran1_file`, `outgofz`
- `symbols.rs`: `setup_input_file`, `read_input_file`, `save_inp_file`; `output.rs`: `r_int`; `symtab_tools.rs`: `VOIDS`

Physical system

Analysed configuration: `200DMSO400Water.ato`, an unchanged copy of the EPSR-test box of 200 dimethyl sulphoxide (DMSO; atom types S, O, C, H) and 400 water molecules (O1, H1): 3200 atoms, DMSO mole fraction 1/3, 300 K, cubic box of side 33.1213 Å (0.0881 atoms/Å^3), the EPSR model of the deuterated DMSO-water neutron data (ISIS SANDALS runs SLS03863, SLS03871, SLS03850), cf. [2]. Distribution 1: all atoms occupy the pixels within 2.0 Å. Distribution 2: radius 2.5 Å with `include O1 H1 XX`: only water atoms occupy pixels and occupied/unoccupied are swapped (`XX`, absent from the `.ato` file, is silently dropped). The grid is adjusted to the radius (99^3 pixels, README); the pixel correlation is sampled from 100 random origins with the Park-Miller generator with Bays-Durham shuffle [3,4].

Inputs

`cavity.VOIDS.dat`: `nsize 100`, `npix 100`, `voidmax 500.0`, `nsumt 0`, the two distributions above (blank `include` for the first); no `randomno.dat`; `200DMSO400Water.ato` copied unchanged from EPSR-test.

How the accuracy was verified

`commands.txt` (1 program call, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in 1 pass.
Single pass. Without `randomno.dat` the generator is reset (idum = 0) and the file is created and saved. Compared: `cavity.VOIDS.n01` (three columns with spreads per distribution), the rewritten `cavity.VOIDS.dat`, `fort.14` and `randomno.dat`. The reference `fort.14` gives fractions of pixels 0.07696 (distribution 1) and 0.62561 (distribution 2).

Notes

Fortran quirks reproduced: the blank `include` value is written back using the word boundaries of the previous keyword (3 blanks), and nsumt and ndist are stored into the `npix` and `voidmax` entries (reference `.dat`: `npix 1`, `voidmax 2`).

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] A. K. Soper and A. Luzar, "A neutron diffraction study of dimethyl sulphoxide-water mixtures", J. Chem. Phys. 97, 1320-1331 (1992).
[3] S. K. Park and K. W. Miller, "Random number generators: good ones are hard to find", Commun. ACM 31, 1192-1201 (1988).
[4] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, Numerical Recipes in Fortran 77, 2nd ed., Cambridge University Press (1992).
[5] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.